In [2]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def purchase_probability(price):
    price = np.asarray(price, dtype=float)
    return (
        0.55 * np.exp(-np.logaddexp(0.0, (price - 13.9) / 0.8))
        + 0.30 * np.exp(-np.logaddexp(0.0, (price - 30.9) / 1.2))
        + 0.15 * np.exp(-np.logaddexp(0.0, (price - 58.4) / 1.8))
    )

def sales(price=14.0, rng=None, mean_customers=30):
    price = float(price)
    rng = np.random.default_rng(rng)

    customer_count = rng.poisson(mean_customers)
    probability = purchase_probability(price)

    return rng.binomial(
        1,
        probability,
        size=customer_count
    ).astype(np.int8)
```

In [3]:
```python
def observe_price(price, mean_customers, stage, rng):
    choices = sales(
        price,
        mean_customers=mean_customers,
        rng=rng
    )

    customer_revenue = price * choices
    n = len(customer_revenue)

    return {
        "stage": stage,
        "price": float(price),
        "customers": n,
        "buyers": int(choices.sum()),
        "revenue": float(customer_revenue.sum()),
        "revenue_per_customer": float(customer_revenue.mean()),
        "standard_error": float(
            customer_revenue.std(ddof=1) / np.sqrt(n)
        )
    }
```

In [11]:
```python
rng = np.random.default_rng(20261104)
```

```
prices = np.arange(0, 70.01, 0.50)
experiments = []

for price in prices:
    purchases = sales(
        price=price,
        mean_customers=1000,
        rng=rng
    )

    experiments.append({
        "price": price,
        "customers": len(purchases),
        "buyers": purchases.sum(),
        "revenue": price * purchases.sum()
    })

results = pd.DataFrame(experiments)

results["revenue_per_customer"] = (
    results["revenue"] / results["customers"]
)
```

In [12]:

```
promising_prices = (
    results.nlargest(10, "revenue_per_customer")["price"]
)

for price in promising_prices:
    new_purchases = sales(
        price=price,
        mean_customers=5000,
        rng=rng
    )

    new_customers = len(new_purchases)
    new_buyers = new_purchases.sum()
    new_revenue = price * new_buyers

    row = results["price"] == price

    # Add the new experiment to the cached totals.
    results.loc[row, "customers"] += new_customers
    results.loc[row, "buyers"] += new_buyers
    results.loc[row, "revenue"] += new_revenue

results["revenue_per_customer"] = (
    results["revenue"] / results["customers"]
)
```

In [18]:
```
rng = np.random.default_rng(20261104)

# Test $0.00, $0.50, $1.00, ..., $70.00
prices = np.arange(0, 70.01, 0.50)
experiment_results = []

for price in prices:
    purchases = sales(
        price=price,
        mean_customers=3000,
        rng=rng
    )

    customers = len(purchases)
    buyers = purchases.sum()
    revenue = price * buyers

    experiment_results.append({
        "price": price,
        "customers": customers,
        "buyers": buyers,
        "revenue": revenue,
        "revenue_per_customer": revenue / customers
    })

results = pd.DataFrame(experiment_results)

best_row = results.loc[
    results["revenue_per_customer"].idxmax()
]

best_price = best_row["price"]
total_consumers = results["customers"].sum()
total_revenue = results["revenue"].sum()

print(f"Selected price: ${best_price:.2f}")
print(f"Total consumers experimented on: {total_consumers:,}")
print(f"Total experimental revenue: ${total_revenue:,.2f}")
print(
    "Revenue per consumer at selected price: "
    f"${best_row['revenue_per_customer']:.3f}"
)
```

```
Selected price: $27.50
Total consumers experimented on: 423,801
Total experimental revenue: $2,718,519.00
Revenue per consumer at selected price: $12.125
```

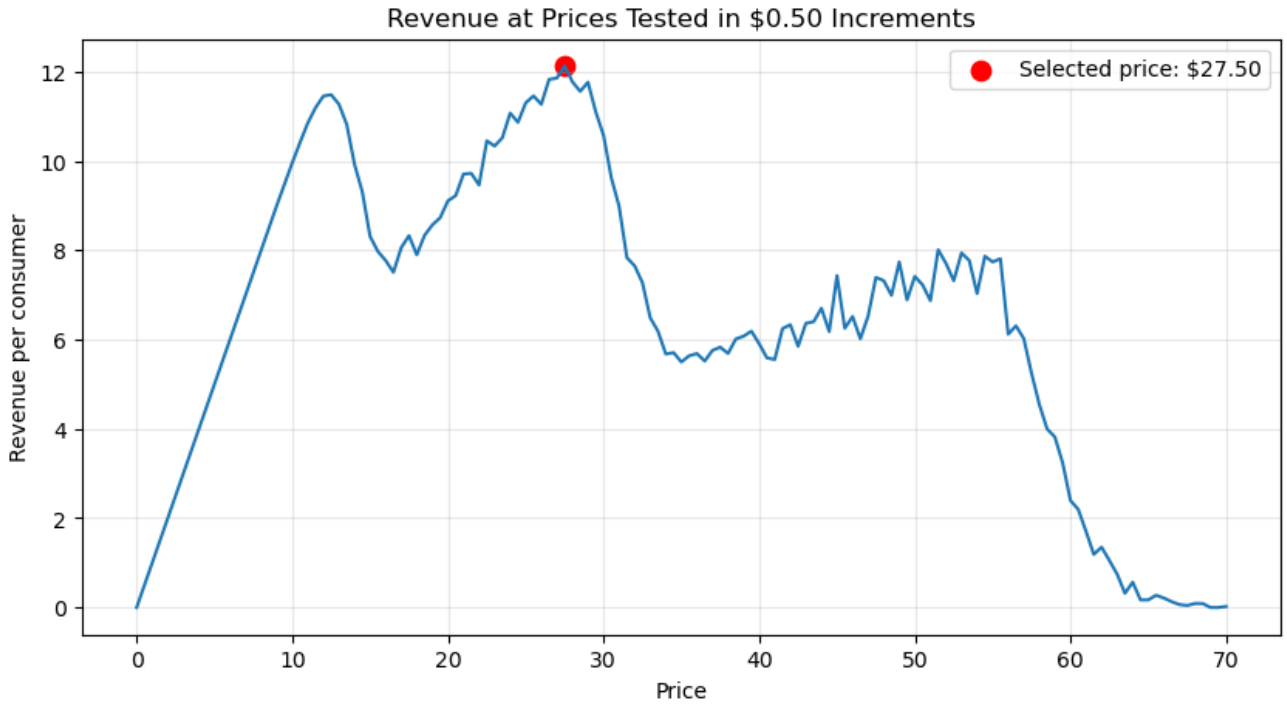
In [19]:
```
plt.figure(figsize=(10, 5))
```

```
plt.plot(
    results["price"],
    results["revenue_per_customer"]
)

plt.scatter(
    best_price,
    best_row["revenue_per_customer"],
    color="red",
    s=80,
    label=f"Selected price: ${best_price:.2f}"
)

plt.xlabel("Price")
plt.ylabel("Revenue per consumer")
plt.title("Revenue at Prices Tested in $0.50 Increments")
plt.legend()
plt.grid(alpha=0.3)

plt.show()
```



Write Up

The goal of this experiment was to find the price between $0 and 70$ that maximizes expected revenue per arriving consumer. Since customer purchase decisions are random, the best price cannot be determined reliably from a single customer or a single comparison. I used an algorithmic grid search that tested prices in $0.50 increments and measured the revenue generated at each price. The algorithm selected the tested price

with the highest observed revenue per consumer while recording the number of customers, buyers, and total revenue from every experiment.

The design handles uncertainty by assigning approximately 3,000 consumers to every price. The exact number varies because the sales function generates customer traffic from a Poisson distribution. Revenue per consumer is calculated by dividing the revenue at a price by the actual number of consumers who saw that price. Using several thousand consumers reduces the effect of unusually high or low purchase counts, although some sampling variation remains. For example, observed revenue at the selected price can exceed the known expected optimum in a particular experiment without implying that the true optimum is higher.

I selected prices through a brute force grid search over the full feasible range. The algorithm tested 141 prices, beginning at 0 dollars and increasing by 50 cents until it reaches 70 dollars. Searching the full range was important because the revenue curve contains multiple local peaks. A local search could stop at a price that appears attractive without finding the global optimum. The 50 cents step size provided enough precision to identify the correct region without requiring experiments at every possible cent.

One challenge was balancing precision against the number of consumers required. Smaller increments provide a more precise price estimate, but they also create more prices to test. Testing every cent would require 7,001 separate price experiments and an impractically large number of consumers. I resolved this problem by using 50 cent increments. This reduced the search to 141 prices while still allowing the algorithm to find $27.50 which is within 25 cents of the most optimal price.

A second challenge was distinguishing genuine revenue differences from random variation. Nearby prices such as $27.50, 28.00, and $27.00 have very similar expected revenue, so the observed winner can change when the experiment is rerun. I addressed this by using 3,000 expected consumers per price and a fixed random seed, which makes the submitted result reproducible. I also stored every experiment in one table so the consumer counts and revenues could be audited and additional observations could be added later if greater precision were required.

The algorithm selected a price of $27.50 after experimenting on $423,801$ consumers. These experiments generated 2,718,519.00 in total revenue, equivalent to 6.415 per experimental consumer. If every experimental consumer had instead gener 11.925, the benchmark total would have been approximately $5,053,827$. Therefore, the experiments captured approximately 53.812 5.125 per

consumer; this is slightly above the $11.925 expected benchmark because of random sampling variation.